

# Chapter 8

## Database Triggers

A trigger is a specification that the database should automatically execute a particular function when- ever a certain type of operation is performed. Triggers can be attached to tables or views.

### 8.1 Steps of Triggers

Creating a trigger in PostgreSQL always requires two steps:

1. **Create a Trigger Function:** A special function written in PL/pgSQL that contains the logic to be executed. Its return type is trigger.
2. **Create the Trigger:** The rule that binds the function to a specific event on **a specific table or database**.

### 8.2 Why Triggers?

Triggers are incredibly powerful tools for backend developers and database administrators. You typically use them for:

- **Auditing and Logging:** Automatically keeping a history of who changed what data and when (e.g., maintaining an audit trail of salary changes).
- **Data Integrity:** Enforcing complex business rules that standard database constraints (CHECK, UNIQUE) cannot handle.
- **Automatic Derivation:** Automatically calculating and updating column values based on changes in other columns (e.g., automatically updating a `last_modified` timestamp or auto ID generation).
- **Security:** Preventing certain database actions entirely under specific conditions.

## 8.3 Triggers in PostgreSQL: Classification

There are **3 types** of triggers in postgres Database:

1. Row-Level Triggers (DML)
2. Statement-Level Triggers (DML)
3. Event-Level Triggers (DDL)

### 8.3.1 Row-Level Triggers

A row-level trigger executes once **for every single row affected by a query**. If you run an UPDATE statement that modifies 20 rows, a row-level trigger will fire 20 times. So, it is a DML trigger.

 **2 Terms:** OLD and NEW When DMLs (INSERT , UPDATE OR DELETE) are issued either the values is NEW or OLD and can be captured in trigger by these keywords.

| DML Operation | NEW            | OLD            |
|---------------|----------------|----------------|
| INSERT        | VALID          | INVALID (null) |
| UPDATE        | VALID          | VALID          |
| DELETE        | INVALID (null) | VALID          |

#### Example Scenario

Suppose we are automating HR of a company. Employees have basic information as well as his current salary. But based on performance the salary can be updated by the authority. But in order to track each change made, a trigger can be used to monitor and store the required information such as emp ID, Name, his old salary (before update), present salary (after update), when it happened and so on.

#### Steps:

1. Create tables
2. Create a trigger function
3. Attach the trigger to the table
4. Test it

**Step 1: Create tables;**

```
CREATE TABLE employees (  
  id      SERIAL PRIMARY KEY,  
  name    TEXT NOT NULL,  
  salary  NUMERIC(10, 2)  
);  
  
CREATE TABLE salary_audit (  
  id          SERIAL PRIMARY KEY,  
  employee_id INT,  
  old_salary  NUMERIC(10, 2),  
  new_salary  NUMERIC(10, 2),  
  changed_at  TIMESTAMP DEFAULT NOW(),  
  changed_by  TEXT DEFAULT CURRENT_USER  
);
```

**Step 2: Create the trigger function**

```
CREATE OR REPLACE FUNCTION log_salary_change()  
RETURNS TRIGGER AS $$  
BEGIN  
  -- Only log if salary actually changed  
  IF NEW.salary IS DISTINCT FROM OLD.salary THEN  
    INSERT INTO salary_audit (employee_id, old_salary, new_salary)  
    VALUES (OLD.id, OLD.salary, NEW.salary);  
  END IF;  
  
  RETURN NEW; -- Required for BEFORE/AFTER row triggers  
END;  
$$ LANGUAGE plpgsql;
```

 **Points to Note:**

- The function **returns TRIGGER**
- Since it will be placed in a row-level trigger it must **return NEW**

**Step 3: Attach the trigger to the table**

```
CREATE TRIGGER trg_salary_audit  
AFTER UPDATE ON employees  
FOR EACH ROW -- This makes it a row-level trigger  
EXECUTE FUNCTION log_salary_change();
```

 **Points to Note:**

- AFTER can be replaced by BEFORE, and UPDATE can be replaced by either of {INSERT,DELETE}
- As for ON employees it can be replaced by any table or view
- The clause FOR EACH ROW makes it a row-level trigger

**Step 4: Test it**

```
INSERT INTO employees (name, salary) VALUES ('Mahmud', 65000), ('Hasan',
25000), ('A Karim', 85000);
```

(anomaly due to untimely insertion, but it works)

```
select * from employees;
```

```
---+-----+-----
2 | Hasan | 25000.00
1 | Mahmud | 70000.00
3 | A Karim | 85000.00
```

```
UPDATE employees SET salary = 82000 WHERE id=1; -- will be logged
```

```
UPDATE employees SET salary = 85000 WHERE id=3; -- will not be logged
```

```
-- log
```

```
select * from salary_audit;
```

```
id | employee_id | old_salary | new_salary | changed_at | changed_by
---+-----+-----+-----+-----+-----
1 | 1 | 70000.00 | 82000.00 | 2026-06-24 15:50:04.672293 | armk
```

## 8.4 Statement-Level Triggers

A statement-level trigger fires exactly once per SQL command, regardless of how many rows are modified. **If an update (also delete) modifies 0 rows, 1 row, or 10,000 rows, a statement-level trigger executes just one time.**

**Note:** It does not have direct access to OLD and NEW row variables

### Example:

```
CREATE TABLE products (
id SERIAL PRIMARY KEY,
name VARCHAR(100),
price NUMERIC
);
```

```
INSERT INTO products (name, price) VALUES
('Pen', 0.50),
('Notebook', 4.50),
('Eraser', 0.25);
```

```
-- 1. Create an audit log table
```

```
CREATE TABLE bulk_audit_log (
log_id SERIAL PRIMARY KEY,
action_type TEXT,
executed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```

-- 2. Create the function
CREATE OR REPLACE FUNCTION log_bulk_operation()
RETURNS TRIGGER AS $$
BEGIN
INSERT INTO bulk_audit_log (action_type)
VALUES (TG_OP); -- TG_OP is a built-in variable (INSERT, UPDATE, DELETE)
RETURN NULL; -- Statement-level triggers always return NULL
END;
$$ LANGUAGE plpgsql;

-- 3. Create the statement-level trigger
CREATE TRIGGER trg_statement_bulk_log
AFTER UPDATE OR DELETE ON products
FOR EACH STATEMENT -- This defines it as a Statement-Level trigger
EXECUTE FUNCTION log_bulk_operation();

--4: now test it
UPDATE products SET price = price + 0.70;

--- now select
select * from bulk_audit_log;

```

**Notes:**

- Statement level trigger always return NULL
- TG\_OP is a built-in variable storing type of DML performed (INSERT, UPDATE, DELETE)

Drop syntax: `drop function log_bulk_operation();`

`DROP trigger trg_statement_bulk_log on products;`

### 8.4.1 Event Trigger

While Row and Statement triggers only care about Data Manipulation Language (DML) like INSERT or UPDATE, Event Triggers catch Data Definition Language (DDL) events. They are database-wide and fire on events like CREATE TABLE, DROP TABLE, or ALTER TABLE.

**One assignment will be posted to log all users history in the database.**

Remove below text

Run it as **postgres super user** but only works as this postgres (will be solved later)

```

CREATE TABLE ddl_audit_log (
id          SERIAL PRIMARY KEY,
event_type TEXT,          -- e.g. 'ddl_command_end'
command_tag TEXT,         -- e.g. 'CREATE TABLE', 'DROP INDEX'
object_type TEXT,         -- e.g. 'table', 'index'
schema_name TEXT,
object_name TEXT,

```

```

executed_by TEXT DEFAULT CURRENT_USER,
executed_at TIMESTAMP DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION log_ddl_event()
RETURNS event_trigger AS $$
DECLARE
obj RECORD;
BEGIN
-- pg_event_trigger_ddl_commands() returns info about what just happened
FOR obj IN SELECT * FROM pg_event_trigger_ddl_commands()
LOOP
INSERT INTO ddl_audit_log (event_type, command_tag, object_type, schema_name,
    object_name)
VALUES (
TG_EVENT,          -- the event name
TG_TAG,            -- the SQL command tag
obj.object_type,
obj.schema_name,
obj.object_identity
);
END LOOP;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE EVENT TRIGGER trg_log_ddl
ON ddl_command_end
EXECUTE FUNCTION log_ddl_event();

GRANT CREATE ON SCHEMA public TO armk;

```

**Notes:**

- The return type is event\_trigger
- **ON** sql\_drop is an event fires when someone attempts to drop any object. There are more events like this such as ddl\_command\_end it fires after each ddl statement is executed.

Drop Syntax: **DROP** EVENT **TRIGGER** trg\_block\_drop;